DevOps Project Report:
Scientific Calculator

Name: Preet Chandrakar

Roll Number: MT2025094

---

1. Introduction to DevOps

DevOps is a set of practices that combines software development (Dev) and IT operations (Ops) to improve collaboration and automate the software delivery process. The main goal of devOps is to develop, test, and release software faster and more reliably. It uses practices like Continuous Integration (CI), Continuous Deployment (CD) and automation to reduce errors and improve efficiency in the software development lifecycle.

Traditional software development relies on manual build, testing, and development processes, which can be slow, inconsistent, and prone to human errors. DevOps addresses these issues by automating repetitive tasks, enabling faster feedback, improving code quality, and ensuring consistent application deployment.

.

This project mainly focuses on building **Scientific Calculator** using modern DevOps practices. The application is built using Java and supports the following operations:

- **Square root function -** $\sqrt{x}$
- **Factorial function -** $x!$
- **Natural logarithm (base e) -** $\ln(x)$
- **Power function -** x^b

2. Tools used

Tools	Purpose
Git & GitHub	Source Code Management
Java	Application logic
JUnit	Unit Testing
Maven	Build Automation & dependency management
Jenkins	Continuous Integration & Delivery (CI/CD)
Docker	Containerization
Docker Hub	Image registry
Ansible	Automated deployment

3. Project Setup

Git Commands

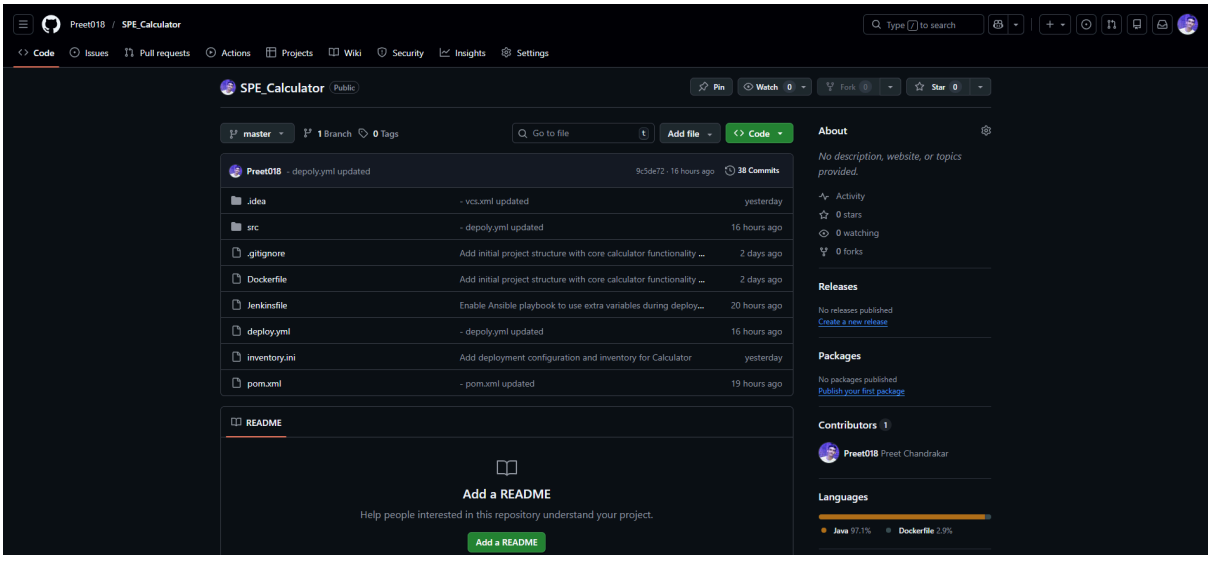
git clone https://github.com/Preet018/SPE_Calculator.git

git add .

git commit -m “Changes”

git push origin master

Screenshot 1: GitHub Repository



4. Testing with JUnit

Test cases:

- Valid edge cases (e.g. factorial of negative number, square root of negative number, etc)
- Parameterized tests for multiple inputs.

Code Snippet: CalculatorTest.java

```java
import org.example.Calculator;

import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

public class CalculatorTest {
    // Square Root Function
    @Test
    public void testSquareRoot() {
        assertEquals( expected: 4.0, Calculator.squareRoot( x: 16),  delta: 0.0001);
        assertEquals( expected: 5.0, Calculator.squareRoot( x: 25),  delta: 0.0001);
    }

    @Test
    public void testSquareRootNegative() {
        assertThrows(IllegalArgumentException.class, () -> {
            Calculator.squareRoot( x: -9);
        });

        assertThrows(IllegalArgumentException.class, () -> {
            Calculator.squareRoot( x: -15);
        });
    }

    // Factorial Function
    @Test
    public void testFactorial() {
        assertEquals( expected: 120.0, Calculator.factorial( x: 5));
        assertEquals( expected: 1.0, Calculator.factorial( x: 0));
    }
```

```java
    @Test
    public void testFactorialNegative() {
        assertThrows(IllegalArgumentException.class, () -> {
            Calculator.factorial( x: -3);
        });

        assertThrows(IllegalArgumentException.class, () -> {
            Calculator.factorial( x: -10);
        });
    }
```

```java
    @Test
    public void testFactorialTooLarge() {
        assertThrows(IllegalArgumentException.class, () -> {
            Calculator.factorial( x: 45);
        });
    }

    // Natural Log Function
    @Test
    public void testNaturalLog() {
        assertEquals(Math.log(10), Calculator.naturalLog( x: 10),  delta: 0.0001);
        assertEquals(Math.log(1), Calculator.naturalLog( x: 1),  delta: 0.0001);
    }

    @Test
    public void testNaturalLogInvalid() {
        assertThrows(IllegalArgumentException.class, () -> {
            Calculator.naturalLog( x: 0);
        });

        assertThrows(IllegalArgumentException.class, () -> {
            Calculator.naturalLog( x: -18);
        });
    }

```

```java
    // Power Function
    @Test
    public void testPower() {
        assertEquals( expected: 32.0, Calculator.power( x: 2,  b: 5),  delta: 0.0001);
        assertEquals( expected: 27.0, Calculator.power( x: 3,  b: 3),  delta: 0.0001);
        assertEquals( expected: 1.0, Calculator.power( x: 5,  b: 0),  delta: 0.0001);
    }

    @Test
    public void testPowerInvalid() {
        assertThrows(IllegalArgumentException.class, () -> {
            Calculator.power( x: 0,  b: -2);
        });

        assertThrows(IllegalArgumentException.class, () -> {
            Calculator.power( x: 0,  b: -5);
        });
    }
}

```

Screenshot 2: JUnit Test Results

```
[INFO] -------------------------------------------------------
[INFO]  T E S T S
[INFO] -------------------------------------------------------
[INFO] Running CalculatorTest
[INFO] Tests run: 9, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.062 s -- in CalculatorTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 9, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO]
[INFO] --- jar:3.3.0:jar (default-jar) @ Calculator ---
[INFO] Building jar: C:\Users\chand\Downloads\IIIT Bangalore Stuffs\Term 2\SPE\Mini Project\Calculator\target\Calculator-1.0-SNAPSHOT.jar
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  3.868 s
[INFO] Finished at: 2026-03-06T15:13:49+05:30
[INFO] ------------------------------------------------------------------------
```

5. Build with Maven

pom.xml configuration:

- Added dependencies for JUnit and Maven plugin.
- Configured <mainClass> for executable JAR.

Maven build command:

mvn clean package

Screenshot 3: Maven Build Output

```
[INFO] Scanning for projects...
[INFO]
[INFO] ----------------------< org.example:Calculator >----------------------
[INFO] Building Calculator 1.0-SNAPSHOT
[INFO]   from pom.xml
[INFO] --------------------------------[ jar ]---------------------------------
[INFO]
[INFO] --- clean:3.2.0:clean (default-clean) @ Calculator ---
[INFO] Deleting C:\Users\chand\Downloads\IIIT Bangalore Stuffs\Term 2\SPE\Mini Project\Calculator\target
[INFO]
[INFO] --- resources:3.3.1:resources (default-resources) @ Calculator ---
[INFO] Copying 0 resource from src\main\resources to target\classes
[INFO]
[INFO] --- compiler:3.8.1:compile (default-compile) @ Calculator ---
[INFO] Changes detected - recompiling the module!
[INFO] Compiling 1 source file to C:\Users\chand\Downloads\IIIT Bangalore Stuffs\Term 2\SPE\Mini Project\Calculator\target\classes
[INFO]
[INFO] --- resources:3.3.1:testResources (default-testResources) @ Calculator ---
[INFO] skip non existing resourceDirectory C:\Users\chand\Downloads\IIIT Bangalore Stuffs\Term 2\SPE\Mini Project\Calculator\src\test\resources
[INFO]
[INFO] --- compiler:3.8.1:testCompile (default-testCompile) @ Calculator ---
[INFO] Changes detected - recompiling the module!
[INFO] Compiling 1 source file to C:\Users\chand\Downloads\IIIT Bangalore Stuffs\Term 2\SPE\Mini Project\Calculator\target\test-classes
[INFO]
[INFO] --- surefire:3.2.5:test (default-test) @ Calculator ---
[INFO] Using auto detected provider org.apache.maven.surefire.junitplatform.JUnitPlatformProvider
[INFO]
```

```
[INFO] -------------------------------------------------------
[INFO]  T E S T S
[INFO] -------------------------------------------------------
[INFO] Running CalculatorTest
[INFO] Tests run: 9, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.062 s -- in CalculatorTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 9, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO]
[INFO] --- jar:3.3.0:jar (default-jar) @ Calculator ---
[INFO] Building jar: C:\Users\chand\Downloads\IIIT Bangalore Stuffs\Term 2\SPE\Mini Project\Calculator\target\Calculator-1.0-SNAPSHOT.jar
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  3.868 s
[INFO] Finished at: 2026-03-06T15:13:49+05:30
[INFO] ------------------------------------------------------------------------
```

6. Continuous Integration with Jenkins

Jenkins Pipeline:

- **Stages:** Checkout → Build → Test → Build Docker Image → Push Docker Image → Ansible Deployment

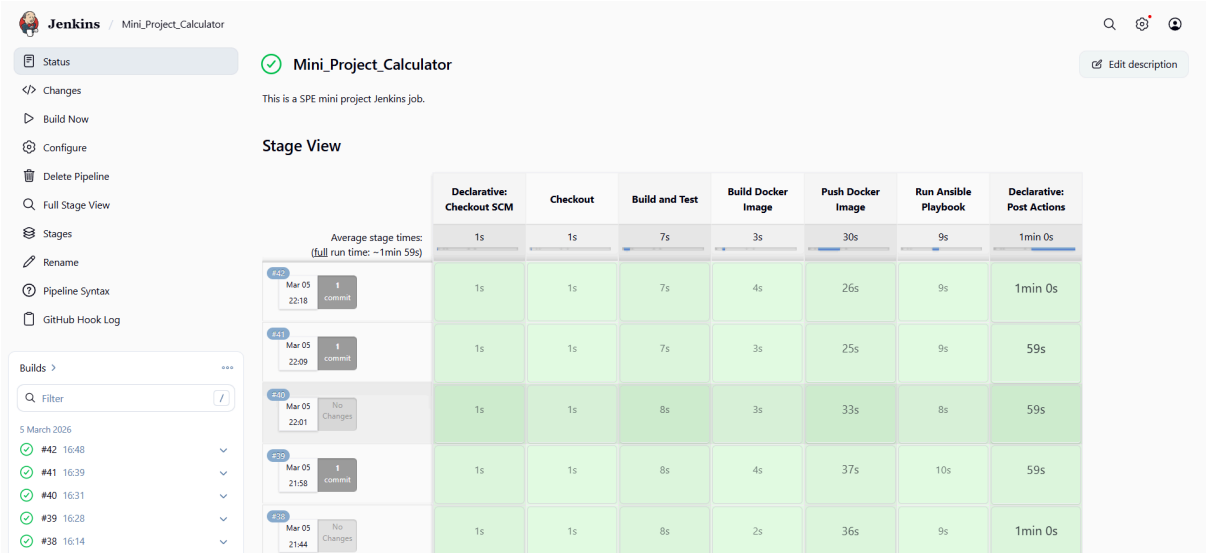
Code Snippet: Jenkinsfile

```
pipeline {
    agent any
    environment {
        DOCKER_IMAGE_NAME = 'scientific-calculator'
        GITHUB_REPO_URL = 'https://github.com/Preet018/SPE_Calculator.git'
        DOCKER_USERNAME = 'preet1018'
        IMAGE_TAG = '0.0.1'
    }
    stages {
        stage('Checkout') {
            steps {
                script {
                    git branch: 'master', url: "${GITHUB_REPO_URL}"
                }
            }
        }
        stage('Build and Test') {
            steps {
                script {
                    sh 'mvn clean install'
                }
            }
        }
        stage('Build Docker Image') {
            steps {
                script {
                    docker.build("${DOCKER_IMAGE_NAME}:${IMAGE_TAG}", '.')
                    // Optionally, also tag the image as "latest" for consistency
                    docker.build("${DOCKER_IMAGE_NAME}:latest", '.')
                }
            }
        }
```

```
        stage('Push Docker Image') {
            steps {
                script {
                    docker.withRegistry('', 'DockerHubCred') {
                        // Tag the image with the appropriate version and latest
                        sh "docker tag ${DOCKER_IMAGE_NAME}:${IMAGE_TAG} ${DOCKER_USERNAME}/scientific-calculator:${IMAGE_TAG}"
                        sh "docker tag ${DOCKER_IMAGE_NAME}:${IMAGE_TAG} ${DOCKER_USERNAME}/scientific-calculator:latest"

                        // Push both the versioned tag and latest tag
                        sh "docker push ${DOCKER_USERNAME}/scientific-calculator:${IMAGE_TAG}"
                        sh "docker push ${DOCKER_USERNAME}/scientific-calculator:latest"
                    }
                }
            }
        }
        stage('Run Ansible Playbook') {
            steps {
                script {
                    ansiblePlaybook(
                        playbook: 'deploy.yml',
                        inventory: 'inventory.ini',
                        extras: '-K'
                    )
                }
            }
        }
    }
```

```
    post {
        success {
            echo 'Pipeline successfully completed!'
            emailext(
                to: 'chandrakarpreet.1100@gmail.com',
                subject: 'Build Success: Scientific Calculator',
                body: 'The Jenkins pipeline for the Scientific Calculator project has completed successfully.'
            )
        }
        failure {
            echo 'Pipeline failed!'
            emailext(
                to: 'chandrakarpreet.1100@gmail.com',
                subject: 'Build Failure: Scientific Calculator',
                body: 'The Jenkins pipeline for the Scientific Calculator project has failed.'
            )
        }
    }
}
```

Screenshot 4: Full jenkins pipeline view



7. Containerization with Docker

Code Snippet: Dockerfile

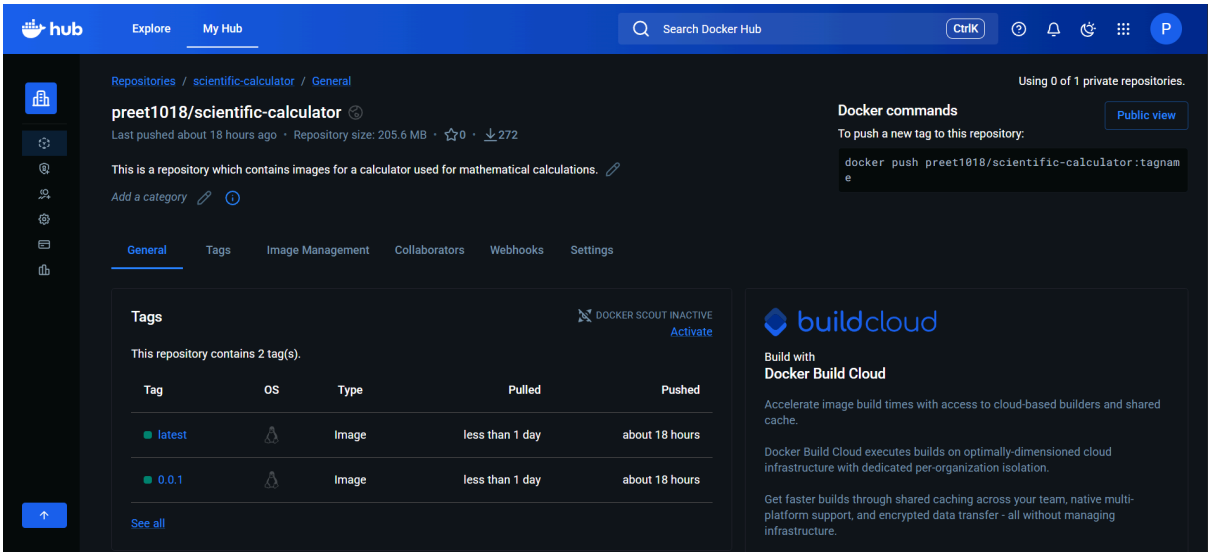
```
FROM amazoncorretto:17

WORKDIR /app

# Copy the built JAR file from the previous stage
COPY target/Calculator-1.0-SNAPSHOT.jar /app/calculator.jar

# Start up cmd to run the application
ENTRYPOINT ["java", "-jar", "calculator.jar"]
```

Docker commands:

docker build preet1018/scientific-calculator:0.0.1 .

docker push preet1018/scientific-calculator:0.0.1

Screenshot 5: Docker image(s) on Docker Hub



8. Deployment with Ansible

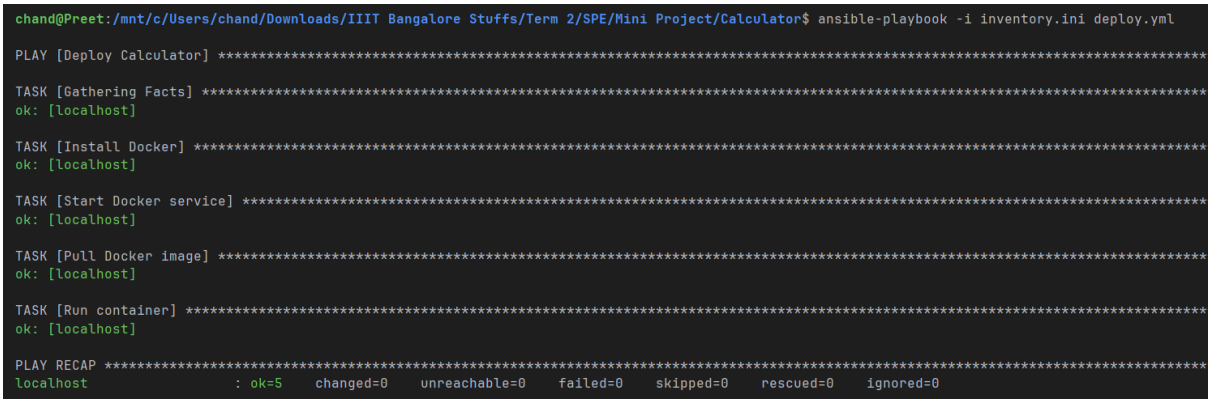
Code Snippet: deploy.yml

```yaml
---
- name: Deploy Calculator
  hosts: localhost
  become: yes # Enable sudo
  become_user: root # Run tasks as root
  tasks:
    - name: Install Docker
      apt:
        name: docker.io
        state: present

    - name: Start Docker service
      service:
        name: docker
        state: started

    - name: Pull Docker image
      docker_image:
        name: preet1018/scientific-calculator:0.0.1
        source: pull

    - name: Run container
      docker_container:
        name: calculator
        image: preet1018/scientific-calculator:0.0.1
        state: started
        restart_policy: always
        interactive: true
```

Code Snippet: inventory.ini

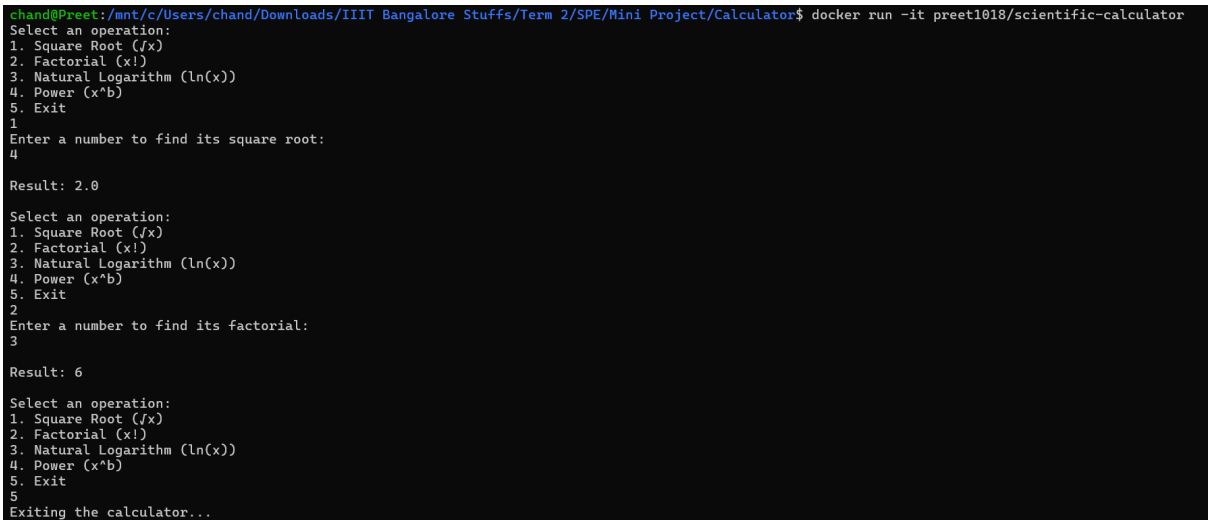
```
[local]
localhost ansible_connection=local ansible_python_interpreter=/usr/bin/python3 ansible_become_pass=9589
```

Screenshot 6: Ansible Deployment Output



9. Application Screenshots

Screenshot 7: Application's Command Line Interface (CLI)



10. Conclusion

This project shows how to create and implement a **Scientific Calculator** application using DevOps practices. The project ensures that the application can be built, tested, packaged, and deployed consistently by incorporating automation into various phases of the software

development lifecycle. The development process is more efficient due to automated pipeline, which also reduces the possibility of human error and manual interactability.

Links:

- GitHub Repository: https://github.com/Preet018/SPE_Calculator
- Docker Repository: https://hub.docker.com/r/preet1018/scientific-calculator